entagile

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Towards a Digital Professor

A course-grounded, multimodal tutoring system

Ovunc Eker Jonathan Ma Zixiang "Sean" Song Yining "Lily" Li

October 2026

Objectives

Digital Professor

Build a digital tutor that understands a course, accepts the ways students naturally show their work, and adapts its support as their understanding changes.

Understand the course

Use syllabi, lecture notes, textbooks, problems, and other approved sources.

Interact naturally

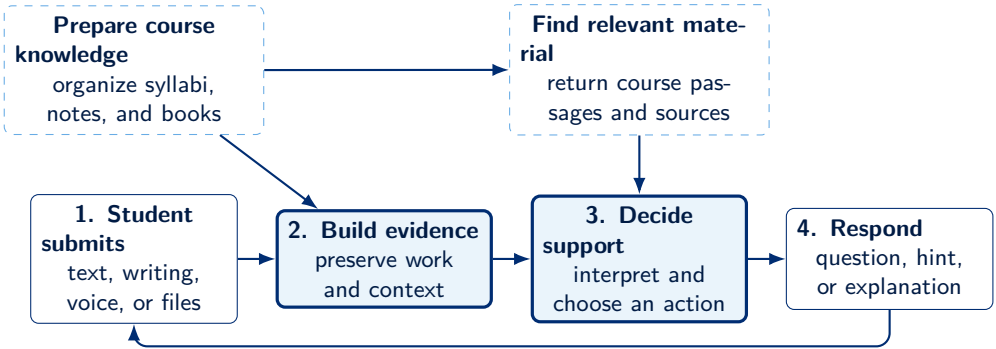
Support text, equations, documents, handwriting, voice, and future live sessions.

Support learning

Ask questions, give hints or explanations, capture revisions, and reassess understanding.

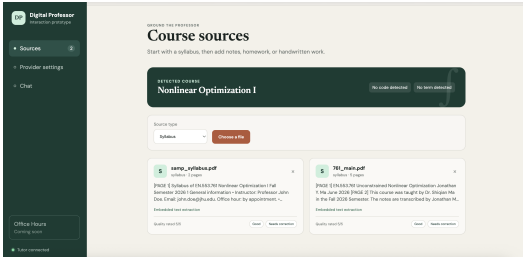
Design goal: reuse the same system across courses while changing the course material and teaching context.

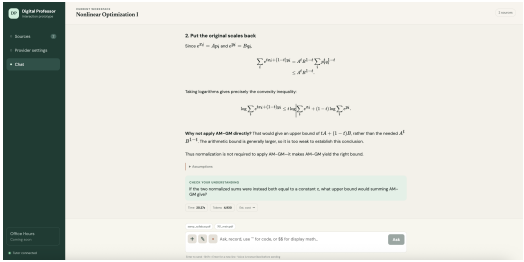
How the system works



The cycle repeats: each student response becomes new evidence, so the system can reassess what help is appropriate instead of treating every message as an isolated question.

Prototype status: what works today





Course material

Upload documents, identify course information, preview extracted content, and flag uncertain results.

Student interaction

Ask through text, equations, handwriting, or recorded voice; view rendered mathematics, cited sources, and request timing.

Current status: a local research prototype demonstrates the main inputs and handoffs; it is not yet a complete adaptive tutor.

Existing products

Compare relevant digital tutoring products in plain language: what students can do with them today, what they do well, and which gaps motivate our approach.

Advanced course ingestion and context

Explain how syllabi, lecture notes, textbooks, and questions become reliable course context. Show what structured information is produced and how the tutoring system uses it.

Course knowledge module: inputs, functions, and outputs

Sean should explain his module as one simple pipeline:

Inputs	Functions	Outputs
List the course documents, structured course context, student question, and any filters the module receives.	Describe how material is divided, stored, searched, ranked, and checked for relevance.	Show the passages, page references, confidence or status, and metadata returned to the tutoring system.

Include one concrete example showing a question entering the module and the course evidence returned downstream.

Risk matrix

Show the few risks most likely to affect the project or student experience. Explain their impact and the practical step planned to reduce each one.

Future work

1. Implement the architecture	2. Build a Calculus test set	3. Review independently
Connect course ingestion, knowledge retrieval, student evidence, tutoring decisions, and delivery through agreed interfaces.	Create representative course materials, student work, and tutoring situations covering correct, partial, incorrect, and uncertain responses.	Have an independent mathematics reviewer verify mathematical correctness, use of course evidence, and the quality of tutoring decisions.

Second-half goal: demonstrate a complete Calculus tutoring loop whose inputs, evidence, decisions, and feedback can all be inspected.

Questions?

5-minute Q&A

Ovunc Eker Jonathan Ma Sean Song Yining Li